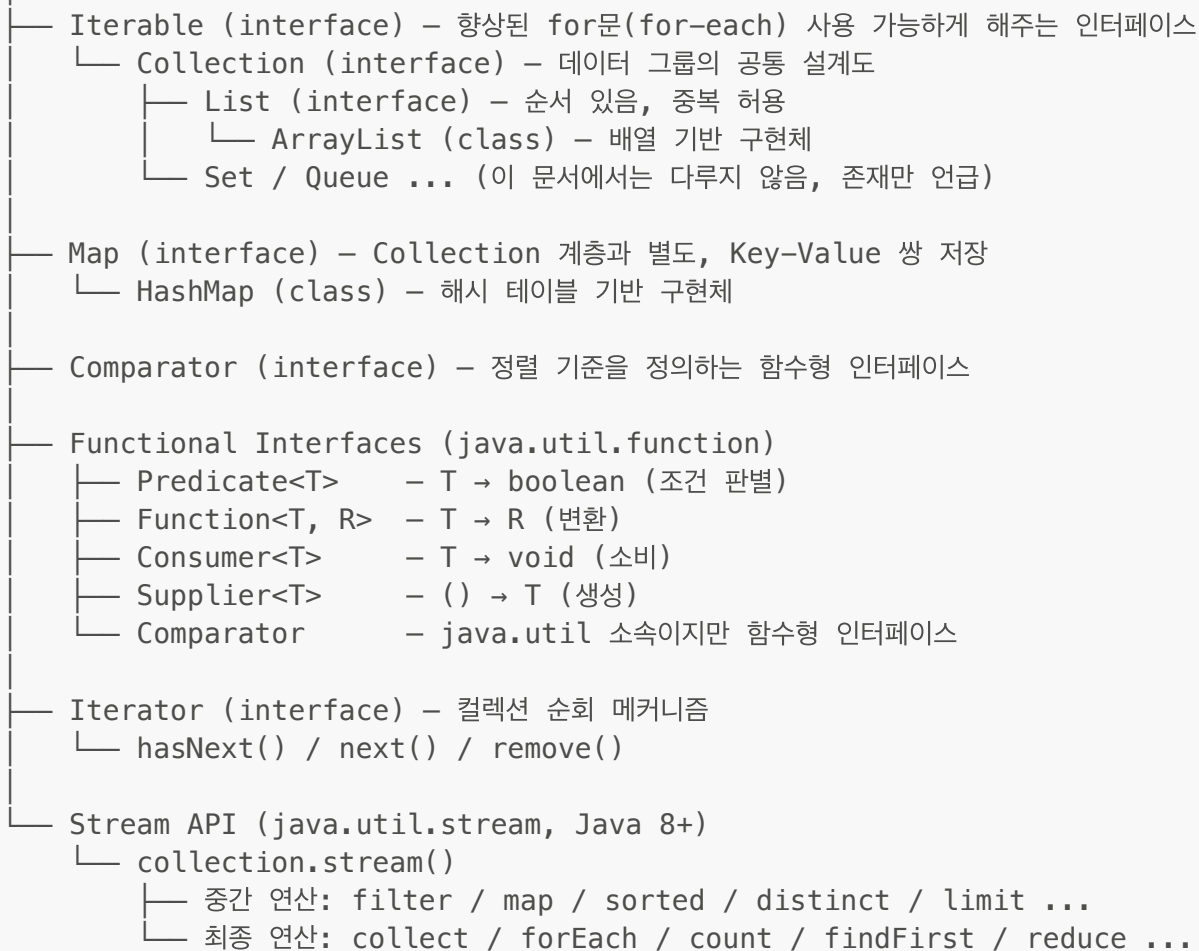


Java Collections Framework & Stream API 전체 구조

코딩테스트에서 자주 쓰는 Java 자료구조와 API의 전체 계층 구조를 한눈에 파악한다.

1. Java Collections Framework & Stream API — 전체 계층도

Java Collections Framework & Stream API



2. 인터페이스를 역할별로 재분류

데이터 구조 설계도

- **Iterable** → **Collection** → **List** / **Set** / **Queue**
- **Map** (Collection과 별도 계층)

행위 설계도 (함수형 인터페이스)

- **Comparator** — 정렬 기준 정의
- **Predicate** — 조건 판별 (true/false)
- **Function** — 값 변환 (입력 → 출력)

- **Consumer** — 값 소비 (리턴 없음)
- **Supplier** — 값 생성 (입력 없음)

순회 메커니즘

- **Iterator** — `hasNext()/next()`로 하나씩 꺼내는 방식
- **Iterable**을 구현하면 향상된 for문 사용 가능 (내부적으로 Iterator 사용)

처리 파이프라인

- **Stream API** — 컬렉션 데이터를 선언적으로 가공하는 파이프라인

3. Java 8 이전 vs 이후 — 왜 Java 8이 중요한가

Java 8 이전

- 정렬할 때 익명 클래스(anonymous class)로 Comparator 구현
- 반복문은 외부 반복(external iteration) — `for (int i = 0; ...)`
- 조건 필터링도 전부 for문 + if문 조합

Java 8에서 추가된 것들

기능	왜 필요했나
람다 표현식	익명 클래스의 보일러플레이트 코드 제거
함수형 인터페이스 (Predicate, Function 등)	람다를 받을 표준화된 타입 필요
Stream API	외부 반복 → 내부 반복, 선언적 데이터 처리
Default 메서드	기존 Collection 인터페이스에 <code>stream()</code> 추가 가능하게 함
Optional	null 처리 개선

비교 1: 리스트 정렬 — 익명 클래스 vs 람다

```
import java.util.*;

public class SortComparison {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(Arrays.asList("Charlie",
        "Alice", "Bob"));

        // === Java 8 이전: 익명 클래스 ===
        Collections.sort(names, new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return a.compareTo(b);
            }
        });
        System.out.println(names); // [Alice, Bob, Charlie]
```

```
// === Java 8 이후: 람다 ===
names = new ArrayList<>(Arrays.asList("Charlie", "Alice", "Bob"));
names.sort((a, b) -> a.compareTo(b));
System.out.println(names); // [Alice, Bob, Charlie]
}
}
```

비교 2: 리스트 필터링 — for문 vs Stream

```
import java.util.*;
import java.util.stream.*;

public class FilterComparison {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9,
10);

        // === Java 8 이전: for문 ===
        List<Integer> evenOld = new ArrayList<>();
        for (int n : numbers) {
            if (n % 2 == 0) {
                evenOld.add(n);
            }
        }
        System.out.println(evenOld); // [2, 4, 6, 8, 10]

        // === Java 8 이후: Stream ===
        List<Integer> evenNew = numbers.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());
        System.out.println(evenNew); // [2, 4, 6, 8, 10]
    }
}
```

비교 3: Map 값 업데이트 — if-else vs getOrDefault/merge

```
import java.util.*;

public class MapUpdateComparison {
    public static void main(String[] args) {
        String[] fruits = {"apple", "banana", "apple", "cherry", "banana",
"apple"};

        // === Java 8 이전: if-else ===
        Map<String, Integer> countOld = new HashMap<>();
        for (String fruit : fruits) {
            if (countOld.containsKey(fruit)) {
                countOld.put(fruit, countOld.get(fruit) + 1);
            }
        }
    }
}
```

```

        } else {
            countOld.put(fruit, 1);
        }
    }
    System.out.println(countOld); // {banana=2, cherry=1, apple=3}

    // === Java 8 이후: getOrDefault ===
    Map<String, Integer> countNew = new HashMap<>();
    for (String fruit : fruits) {
        countNew.put(fruit, countNew.getOrDefault(fruit, 0) + 1);
    }
    System.out.println(countNew); // {banana=2, cherry=1, apple=3}

    // === Java 8 이후: merge ===
    Map<String, Integer> countMerge = new HashMap<>();
    for (String fruit : fruits) {
        countMerge.merge(fruit, 1, Integer::sum);
    }
    System.out.println(countMerge); // {banana=2, cherry=1, apple=3}
}
}

```

4. Iterator 심화

Iterator 인터페이스 메서드

메서드	설명
<code>hasNext()</code>	다음 요소가 있으면 true
<code>next()</code>	다음 요소 반환
<code>remove()</code>	<code>next()</code> 로 반환된 마지막 요소 제거

Iterable과의 관계

- **Iterable** 인터페이스를 구현하면 `iterator()` 메서드를 제공해야 함
- 이를 구현한 클래스는 향상된 for문(for-each) 사용 가능

향상된 for문은 내부적으로 Iterator를 사용한다

```

import java.util.*;

public class IteratorInternal {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("A", "B", "C");

        // 향상된 for문
        for (String s : list) {
            System.out.print(s + " "); // A B C
        }
    }
}

```

```

    }
    System.out.println();

    // 위 코드는 컴파일러가 아래와 같이 변환한다
    Iterator<String> it = list.iterator();
    while (it.hasNext()) {
        String s = it.next();
        System.out.print(s + " "); // A B C
    }
    System.out.println();
}
}

```

ConcurrentModificationException — 순회 중 수정 문제

```

import java.util.*;

public class ConcurrentModExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C",
"D"));

        // === 잘못된 방법: for-each 안에서 remove → 예외 발생 ===
        // for (String s : list) {
        //     if (s.equals("B")) list.remove(s); //
ConcurrentModificationException!
        // }

        // === 해결 1: Iterator의 remove() 사용 ===
        Iterator<String> it = list.iterator();
        while (it.hasNext()) {
            if (it.next().equals("B")) {
                it.remove(); // 안전하게 제거
            }
        }
        System.out.println(list); // [A, C, D]

        // === 해결 2: removeIf() 사용 (Java 8+) ===
        list = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
        list.removeIf(s -> s.equals("B"));
        System.out.println(list); // [A, C, D]
    }
}

```

코딩테스트 관련: Iterator를 직접 쓸 일은 거의 없지만, "for-each 안에서 remove하면 왜 에러가 나는가"를 이해하는 데 핵심 개념이다.

5. 함수형 인터페이스 심화

@FunctionalInterface란?

- 추상 메서드가 **딱 1개**인 인터페이스
- 이 어노테이션을 붙이면 컴파일러가 추상 메서드가 1개인지 검증해준다
- 람다 표현식의 타겟 타입으로 사용 가능

핵심 4가지 + Comparator

인터페이스	추상 메서드	용도	Stream에서 사용처
<code>Predicate<T></code>	<code>boolean test(T t)</code>	조건 판별	<code>stream.filter()</code>
<code>Function<T, R></code>	<code>R apply(T t)</code>	값 변환	<code>stream.map()</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	값 소비	<code>stream.forEach()</code>
<code>Supplier<T></code>	<code>T get()</code>	값 생성	<code>stream.collect()</code> 내부, <code>Optional.orElseGet()</code>
<code>Comparator<T></code>	<code>int compare(T a, T b)</code>	정렬 기준	<code>stream.sorted()</code> , <code>list.sort()</code>

Comparator가 함수형 인터페이스인 이유

- 추상 메서드가 `compare` 딱 1개 (`equals`는 `Object`에서 상속받으므로 제외)
- 따라서 람다로 작성 가능: `(a, b) -> a - b`

실무 한 줄 시나리오

```
import java.util.*;
import java.util.function.*;

public class FunctionalInterfaceExamples {
    public static void main(String[] args) {
        // Predicate - 성인 여부 판별
        Predicate<Integer> isAdult = age -> age >= 18;
        System.out.println(isAdult.test(20)); // true
        System.out.println(isAdult.test(15)); // false

        // Function - 이름에서 성만 추출
        Function<String, String> getLastName = name -> name.split(" ")[0];
        System.out.println(getLastName.apply("김 철수")); // 김

        // Consumer - 로그 출력
        Consumer<String> logger = msg -> System.out.println("[LOG] " +
msg);
        logger.accept("서버 시작"); // [LOG] 서버 시작

        // Supplier - 랜덤 ID 생성
        Supplier<String> idGenerator = () -> "ID-" + (int)(Math.random() *

```

```
10000);  
    System.out.println(idGenerator.get()); // ID-xxxx (랜덤)  
  
    // Comparator - 문자열 길이순 정렬  
    List<String> words = new ArrayList<>(Arrays.asList("banana",  
"fig", "apple"));  
    words.sort(Comparator.comparingInt(String::length));  
    System.out.println(words); // [fig, apple, banana]  
    }  
}
```